

Cuadernillo Semestral de Actividades

Actualizado: 4 de noviembre de 2024

El presente cuadernillo posee un compilado con todos los ejercicios que se usarán durante el semestre en la asignatura. Los ejercicios están organizados en forma secuencial, siguiendo los contenidos que se van viendo en la materia.

Cada semana les indicaremos cuáles son los ejercicios en los que deberían enfocarse para estar al día y algunos de ellos serán discutidos en la explicación de práctica.

Recomendación importante:

Los contenidos de la materia se incorporan y fijan mejor cuando uno intenta aplicarlos - **no alcanza con ver un ejercicio resuelto por alguien más**. Para sacar el máximo provecho de los ejercicios, es importante que asistan a las consultas de práctica habiendo intentado resolverlos (tanto como les sea posible). De esa manera podrán hacer consultas más enfocadas y el docente podrá darles mejor feedback.

Ejercicio 1: WallPost

Primera parte

Se está construyendo una red social como Facebook o Twitter. Debemos definir una clase Wallpost con los siguientes atributos: un texto que se desea publicar, cantidad de likes ("me gusta") y una marca que indica si es destacado o no. La clase es subclase de Object.

Para realizar este ejercicio, utilice el recurso que se encuentra en el sitio de la cátedra (o que puede descargar desde [acá](#)). Para importar el proyecto, siga los pasos explicados en el documento "*Trabajando con proyectos Maven, importar un proyecto*". Allí verá que existe la interface Wallpost y la clase WallpostImpl que implementa la interfaz anterior. Una vez importado, dentro del mismo, debe completar la clase WallPostImpl para que entienda:

```
/*
 * Permite construir una instancia del WallpostImpl.
 * Luego de la invocación, debe tener como texto: "Undefined post",
 * no debe estar marcado como destacado y la cantidad de "Me gusta" debe ser 0.
 */
public WallPostImpl()
```

E implemente el protocolo definido en la interfaz Wallpost como se detalla a continuación

```
/*
 * Retorna el texto descriptivo de la publicación
 */
public String getText()

/*
 * Asigna el texto descriptivo de la publicación
 */
public void setText (String descriptionText)

/*
 * Retorna la cantidad de "me gusta"
 */
public int getLikes()

/*
 * Incrementa la cantidad de likes en uno.
 */
public void like()

/*
 * Decrementa la cantidad de likes en uno. Si ya es 0, no hace nada.
 */
public void dislike()

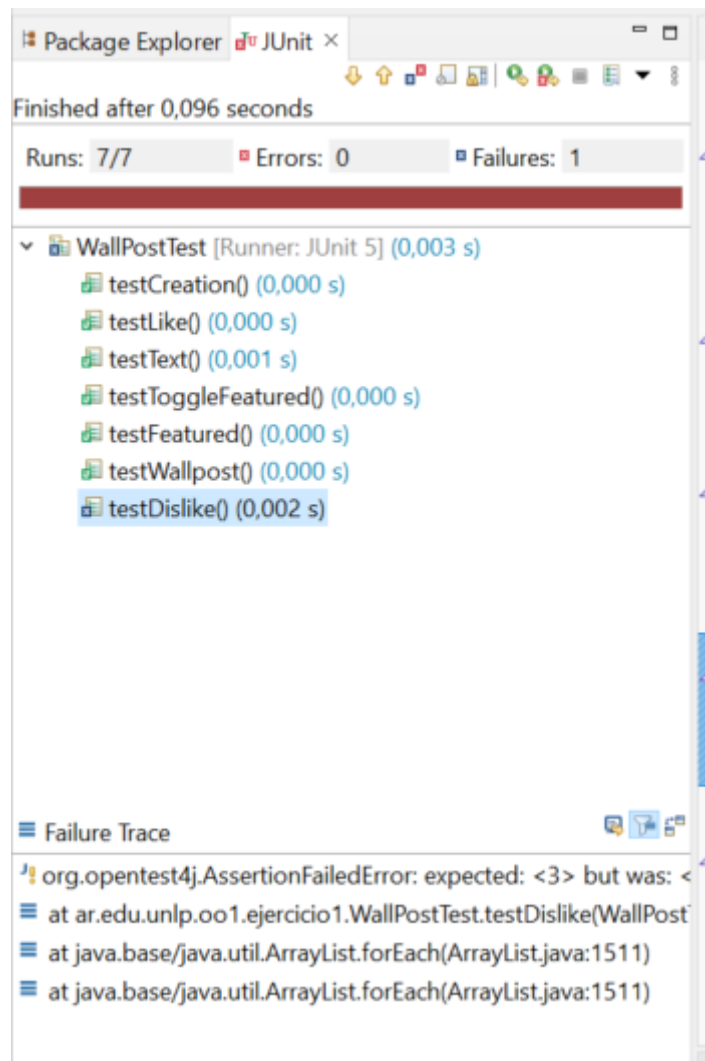
/*
 * Retorna true si el post está marcado como destacado, false en caso contrario
 */
public boolean isFeatured()

/*
 * Cambia el post del estado destacado a no destacado y viceversa.
 */
public void toggleFeatured()
```

Segunda parte

Utilice los tests provistos por la cátedra para comprobar que su implementación de Wallpost es correcta. Estos se encuentran en el mismo proyecto, en la carpeta test, clase WallPostTest.

Para ejecutar los tests simplemente haga click derecho sobre el proyecto y utilice la opción Run As >> JUnit Test. Al ejecutarlo, se abrirá una ventana con el resultado de la evaluación de los tests. Siéntase libre de investigar la implementación de la clase de test. Ya veremos en detalle cómo implementarlas.



En el informe, Runs indica la cantidad de test que se ejecutaron. En Errors se indica la cantidad que dieron error y en Failures se indica la cantidad que tuvieron alguna falla, es decir, los resultados no son los esperados. Abajo, se muestra el Failure Trace del test que falló. Si lo selecciona, mostrará el mensaje de error correspondiente a ese test, que le ayudará a encontrar la falla. Si hace click sobre alguno de los test, se abrirá su implementación en el editor.

Tercera parte

Una vez que su implementación pasa los tests de la primera parte puede utilizar la ventana que se muestra a continuación, la cual permite inspeccionar y manipular el post (definir su texto, hacer like / dislike y marcarlo como destacado).



Para visualizar la ventana, sobre el proyecto, usar la opción del menú contextual Run As >> Java Application. La ventana permite cambiar el texto del post, incrementar la cantidad de likes, etc. El botón Print to Console imprimirá los datos del post en la consola.

Ejercicio 2: Balanza Electrónica

En el taller de programación ud programó una balanza electrónica. Volveremos a programarla, con algún requerimiento adicional.

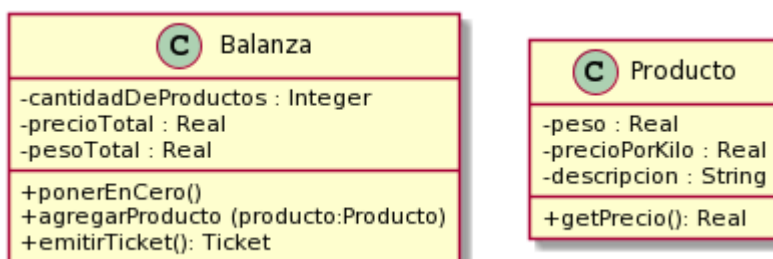
En términos generales, la Balanza electrónica recibe productos (uno a uno), y calcula dos totales: peso total y precio total. Además, la balanza puede poner en cero todos sus valores.

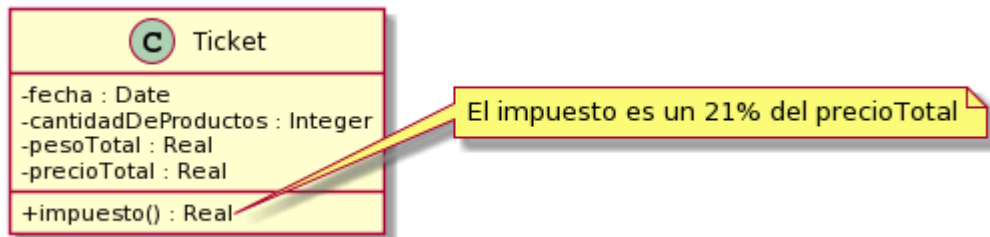
La balanza no guarda los productos. Luego emite un ticket que indica el número de productos considerados, peso total, precio total.

Tareas:

a) Implemente:

Cree un nuevo proyecto Maven llamado `balanzaElectronica`, siguiendo los pasos del documento “*Trabajando con proyectos Maven, crear un proyecto Maven nuevo*”. En el paquete correspondiente, programe las clases que se muestran a continuación.





Observe que no se documentan en el diagrama los mensajes que nos permiten obtener y establecer los atributos de los objetos (accessors). Aunque no los incluimos, verá que los tests fallan si no los implementa. Consulte con el ayudante para identificar, a partir de los tests que fallan, cuales son los accessors necesarios (pista: todos menos los setters de balanza).

Todas las clases son subclasses de Object.

Nota: Para las fechas, utilizaremos la clase `java.time.LocalDate`. Para crear la fecha actual, puede utilizar `LocalDate.now()`. También es posible crear fechas distintas a la actual. Puede investigar más sobre esta clase en

<https://docs.oracle.com/javase/8/docs/api/java/time/LocalDate.html>

b) Probando su implementación:

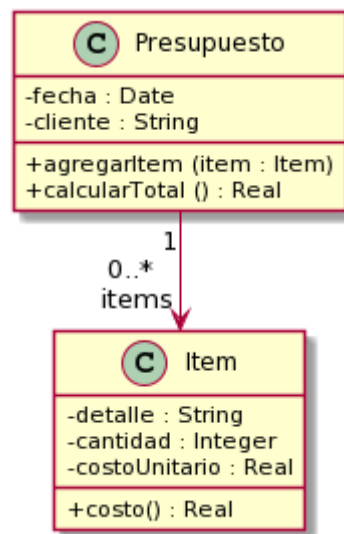
Para realizar este ejercicio, utilice el recurso que se encuentra en el sitio de la cátedra o que puede descargar desde este [link](#). En este caso, se trata de dos clases, `BalanzaTest` y `ProductoTest`, las cuales debe agregar dentro del paquete tests. Haga las modificaciones necesarias para que el proyecto no tenga errores.

Si todo salió bien, su implementación debería pasar las pruebas que definen las clases agregadas en el paso anterior. El propósito de estas clases es ejercitar una instancia de la clase `Balanza` y verificar que se comporta correctamente.

Ejercicio 3: Presupuestos

Un presupuesto se utiliza para detallar los precios de un conjunto de productos que se desean adquirir. Se realiza para una fecha específica y es solicitado por un cliente, proporcionando una visión de los costos asociados.

El siguiente diagrama muestra un diseño para este dominio.



Tareas:

a) Implemente:

Defina el proyecto Ejercicio 3 - Presupuesto y dentro de él implemente las clases que se observan en el diagrama. Ambas son subclases de Object.

b) Discuta y reflexione

Preste atención a los siguientes aspectos:

- ¿Cuáles son las variables de instancia de cada clase?
- ¿Qué variables inicializa? ¿De qué formas se puede realizar esta inicialización?
- ¿Qué ventajas y desventajas encuentra en cada una de ellas?

c) Probando su código:

Utilice los [tests provistos](#) para confirmar que su implementación ofrece la funcionalidad esperada. En este caso, se trata de dos clases: ItemTest y PresupuestoTest, que debe agregar dentro del paquete tests. Haga las modificaciones necesarias para que el proyecto no tenga errores. Siéntase libre de explorar las clases de test para intentar entender qué es lo que hacen.

Ejercicio 4: Balanza mejorada

Realizando el ejercicio de los presupuestos, aprendimos que un objeto puede tener una colección de otros objetos. Con esto en mente, ahora queremos mejorar la balanza implementada en el ejercicio 2.

Tarea 1

Mejorar la balanza para que recuerde los productos ingresados (los mantenga en una colección). Analice de qué forma puede realizarse este nuevo requerimiento e implemente el mensaje

```
public List<Producto> getProductos()
```

que retorna todos los productos ingresados a la balanza (en la compra actual, es decir, desde la última vez que se la puso a cero).

¿Qué cambio produce este nuevo requerimiento en la implementación del mensaje `ponerEnCero()` ?

¿Es necesario, ahora, almacenar los totales en la balanza? ¿Se pueden obtener estos valores de otra forma?

Tarea 2

Con esta nueva funcionalidad, podemos enriquecer al Ticket, haciendo que él también conozca a los productos (a futuro podríamos imprimir el detalle). Ticket también debería entender el mensaje `public List<Producto> getProductos()`.

- ¿Qué cambios cree necesarios en Ticket para que pueda conocer a los productos?
- ¿Estos cambios modifican las responsabilidades ya asignadas de realizar cálculo del precio total?. ¿El ticket adquiere nuevas responsabilidades que antes no tenía?

Tarea 3

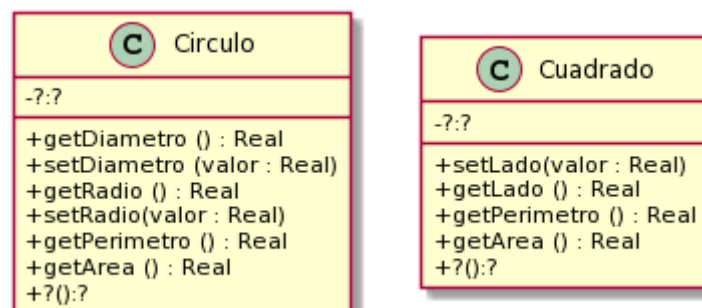
Después de hacer estos cambios, ¿siguen pasando los tests? ¿Está bien que sea así?

Ejercicio 5: Figuras y cuerpos

Figuras en 2D

En Taller de Programación definió clases para representar figuras geométricas. Retomaremos ese ejercicio para trabajar con Cuadrados y Círculos.

El siguiente diagrama de clases documenta los mensajes que estos objetos deben entender.



Fórmulas y mensajes útiles:

- Diámetro del círculo: $\text{radio} * 2$
- Perímetro del círculo: $\pi * \text{diámetro}$
- Área del círculo: $\pi * \text{radio}^2$

- π se obtiene enviando el mensaje #pi a la clase Float (Float pi) (ahora Math.PI)

Tareas:

a) Implementación:

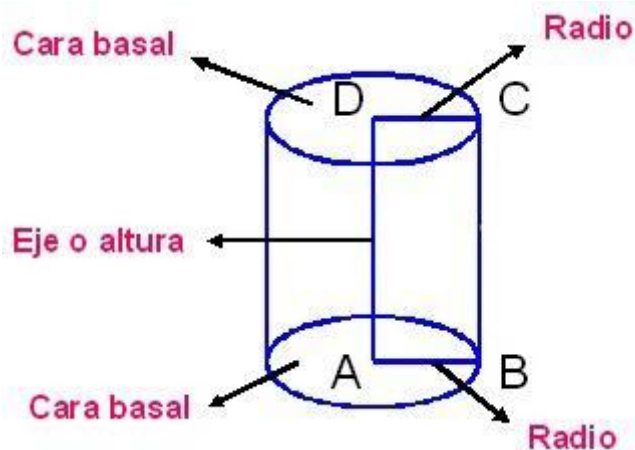
Defina un nuevo proyecto figurasYCuerpos. Implemente las clases Circulo y Cuadrado, siendo ambas subclases de Object. Decida usted qué variables de instancia son necesarias. Puede agregar mensajes adicionales si lo cree necesario.

b) Discuta y reflexione

¿Qué variables de instancia definió? ¿Pudo hacerlo de otra manera? ¿Qué ventajas encuentra en la forma en que lo realizó?

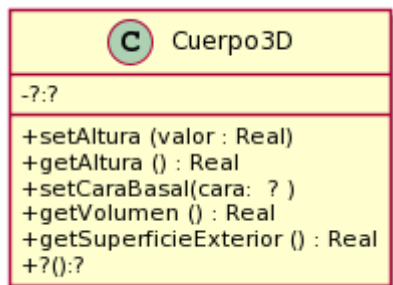
Cuerpos en 3D

Ahora que tenemos Círculos y Cuadrados, podemos usarlos para construir cuerpos (en 3D) y calcular su volumen y superficie o área exterior. Vamos a pensar a un cilindro como "un cuerpo que tiene una figura 2D como cara basal y que tiene una altura (vea la siguiente imagen)". Si en el lugar de la figura2D tuviera un círculo, se formaría el siguiente cuerpo 3D.



Si reemplazamos la cara basal por un rectángulo, tendremos un prisma (una caja de zapatos).

El siguiente diagrama de clases documenta los mensajes que entiende un cuerpo3D.



Fórmulas útiles:

- El área o superficie exterior de un cuerpo es:
 $2 * \text{área-cara-basal} + \text{perímetro-cara-basal} * \text{altura-del-cuerpo}$
- El volumen de un cuerpo es: $\text{área-cara-basal} * \text{altura}$

Más info interesante: A la figura que da forma al cuerpo (el círculo o el cuadrado en nuestro caso) se le llama directriz. Y a la recta en la que se mueve se llama generatriz. En [wikipedia \(Cilindro\)](https://es.wikipedia.org/wiki/Cilindro)¹ se puede aprender un poco más al respecto.

Tareas:

a) Implementación

Implemente la clase Cuerpo 3D, la cuál es subclase de Object. Decida usted qué variables de instancia son necesarias. También decida si es necesario hacer cambios en las figuras 2D.

b) Pruebas automatizadas

Siguiendo los ejemplos de ejercicios anteriores, ejecute [las pruebas automatizadas provistas](#). En este caso, se trata de tres clases (CuerpoTest, TestCirculo y TestCuadrado) que debe agregar dentro del paquete tests. Haga las modificaciones necesarias para que el proyecto no tenga errores. Si algún test no pasa, consulte al ayudante.

c) Discuta y reflexione

Discuta con el ayudante sus elecciones de variables de instancia y métodos adicionales. ¿Es necesario todo lo que definió?

Ejercicio 6: Genealogía salvaje

En una reserva de vida salvaje (como la estación de cría ECAS, en el camino Centenario), los cuidadores quieren llevar registro detallado de los animales que cuidan y sus familias. Para ello nos han pedido ayuda. Debemos:

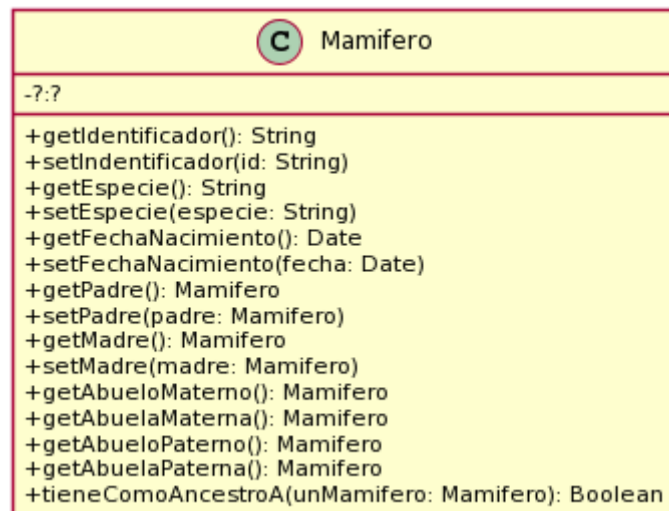
Tareas:

a) Complete el diseño e implemente

Modelar una solución en objetos e implementar la clase Mamífero (como subclase de Object). El siguiente diagrama de clases (incompleto) nos da una idea de los mensajes que un mamífero entiende.

Proponga una solución para el método *tieneComoAncestroA(...)* y *deje la implementación para el final y discuta su solución con el ayudante.*

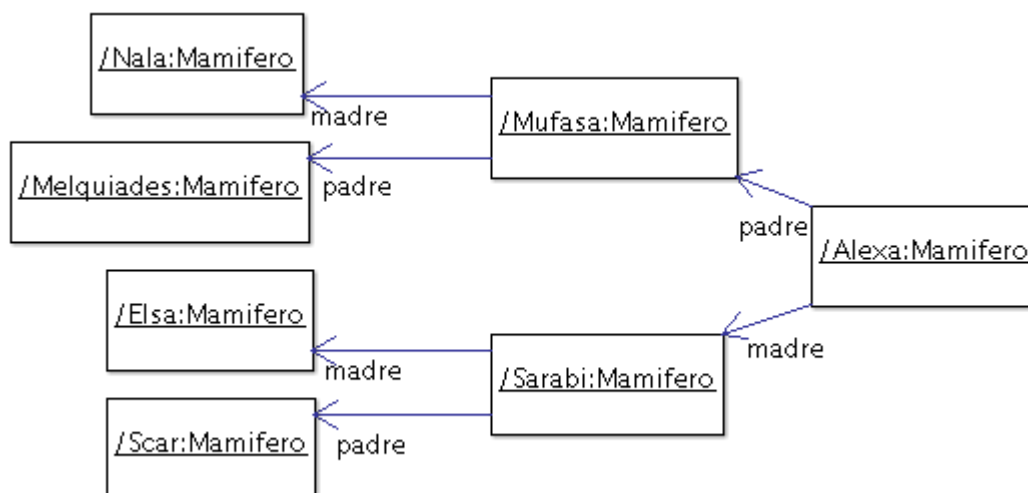
¹ <https://es.wikipedia.org/wiki/Cilindro>



Complete el diagrama de clases para reflejar los atributos y relaciones requeridas en su solución.

b) Pruebas automatizadas

Siguiendo los ejemplos de ejercicios anteriores, ejecute [las pruebas automatizadas provistas](#). En este caso, se trata de una clase, MamiferoTest, que debe agregar dentro del paquete tests. En esta clase se trabaja con la familia mostrada en la siguiente figura.



En el diagrama se puede apreciar el nombre/identificador de cada uno de ellos (por ejemplo Nala, Mufasa, Alexa, etc).

Haga las modificaciones necesarias para que el proyecto no tenga errores. Si algún test no pasa, consulte al ayudante.

Ejercicio 7: Red de Alumbrado

Imagine una red de alumbrado donde cada farola está conectada a una o varias vecinas formando un [grafo conexo](https://es.wikipedia.org/wiki/Grafo_conexo)². Cada una de las farolas tiene un interruptor. Es suficiente con encender o apagar una farola cualquiera para que se enciendan o apaguen todas las demás. Sin embargo, si se intenta apagar una farola apagada (o si se intenta encender una farola encendida) no habrá ningún efecto, ya que no se propagará esta acción hacia las vecinas.

La funcionalidad a proveer permite:

1. crear farolas (inicialmente están apagadas)
2. conectar farolas a tantas vecinas como uno quiera (las conexiones son bi-direccionales)
3. encender una farola (y obtener el efecto antes descrito)
4. apagar una farola (y obtener el efecto antes descrito)

Tareas:

a) Modele e implemente

1. Realice el diagrama UML de clases de la solución al problema.
2. Implemente en Java, la clase Farola, como subclase de Object, con los siguientes métodos:

```
/*
 * Crear una farola. Debe inicializarla como apagada
 */
public Farola ()

/*
 * Crea la relación de vecinos entre las farolas. La relación de vecinos
 * entre las farolas es recíproca, es decir el receptor del mensaje será vecino
 * de otraFarola, al igual que otraFarola también se convertirá en vecina del
 * receptor del mensaje
 */
public void pairWithNeighbor( Farola otraFarola )

/*
 * Retorna sus farolas vecinas
 */
public List<Farola> getNeighbors ()

/*
 * Si la farola no está encendida, la enciende y propaga la acción.
 */
public void turnOn()

/*
 * Si la farola no está apagada, la apaga y propaga la acción.
 */
public void turnOff()
```

² https://es.wikipedia.org/wiki/Grafo_conexo

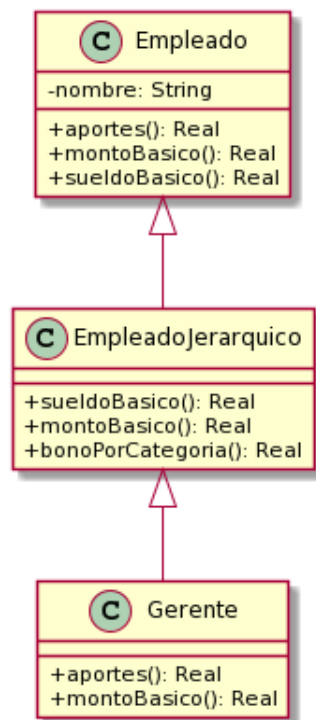
```
/*
 * Retorna true si la farola está encendida.
 */
public boolean isOn()
```

b) Verifique su solución con las pruebas automatizadas

Utilice los [tests provistos](#) por la cátedra para probar las implementaciones del punto 2.

Ejercicio 8: Method lookup con Empleados

Sea la jerarquía de `Empleado` como muestra la figura de la izquierda, cuya implementación de referencia se incluye en la tabla de la derecha.



Empleado	EmpleadoJerarquico	Gerente
<pre>public double montoBasico() { return 35000; }</pre>	<pre>public double sueldoBasico() { return super.sueldoBasico()+ this.bonoPorCategoria(); }</pre>	<pre>public double aportes() { return this.montoBasico() * 0.05d; }</pre>
<pre>public double aportes(){ return 13500; }</pre>	<pre>public double montoBasico() { return 45000; }</pre>	<pre>public double montoBasico() { return 57000; }</pre>
<pre>public double sueldoBasico() { return this.montoBasico() + this.aportes();}</pre>	<pre>public double bonoPorCategoria() { return 8000; }</pre>	

Analice cada uno de los siguientes fragmentos de código y resuelva las tareas indicadas abajo:

```
Gerente alan = new Gerente("Alan Turing");
double aportesDeAlan = alan.aportes();
```

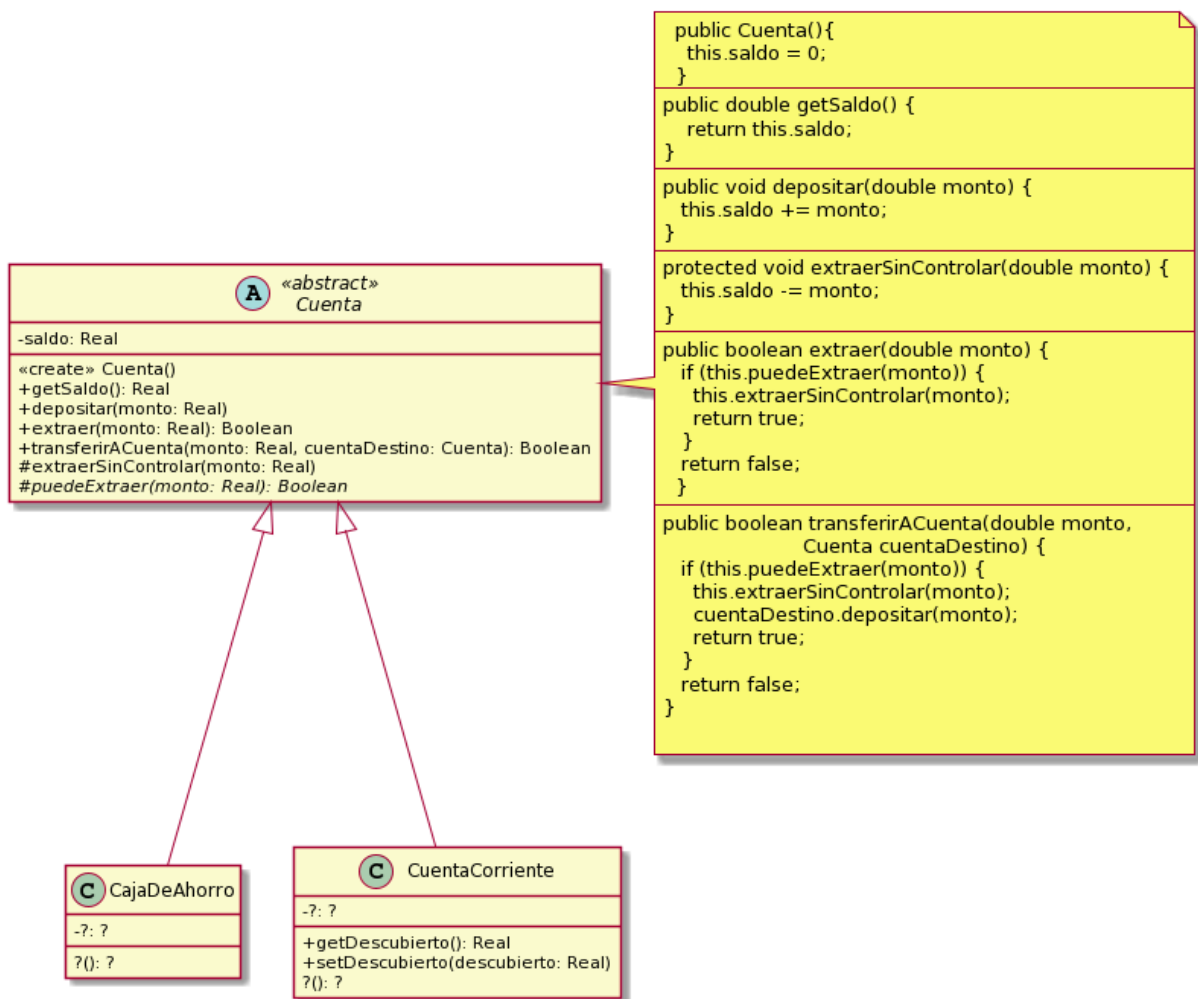
```
Gerente alan = new Gerente("Alan Turing");
double sueldoBasicoDeAlan = alan.sueldoBasico();
```

Tareas:

1. Liste todos los métodos, indicando nombre y clase, que son ejecutados como resultado del envío del último mensaje de cada fragmento de código (por ejemplo, (1) método +aportes de la clase Empleado, (2) ...)
2. ¿Qué valores tendrán las variables aportesDeAlan y sueldoBasicoDeAlan luego de ejecutar cada fragmento de código?

Ejercicio 9: Cuenta con ganchos

Observe con detenimiento el diseño que se muestra en el siguiente diagrama. La clase cuenta es *abstracta*. El método `puedeExtraer()` es abstracto. Las clases `CajaDeAhorro` y `CuentaCorriente` son concretas y están incompletas.



Tarea A: Complete la implementación de las clases `CajaDeAhorro` y `CuentaCorriente` para que se puedan efectuar depósitos, extracciones y transferencias teniendo en cuenta los siguientes criterios.

- 1) Las **cajas de ahorro** solo pueden extraer y transferir cuando cuentan con fondos suficientes.
- 2) Las extracciones, los depósitos y las transferencias desde **cajas de ahorro** tienen un costo adicional de 2% del monto en cuestión (téngalo en cuenta antes de permitir una extracción o transferencia desde caja de ahorro).
- 3) Las **cuentas corrientes** pueden extraer aún cuando el saldo de la cuenta sea insuficiente. Sin embargo, no deben superar cierto límite por debajo del saldo. Dicho límite se conoce como límite de descubierto (algo así como el máximo saldo negativo permitido). Ese límite es diferente para cada cuenta (lo negocia el cliente con la gente del banco).
- 4) Cuando se abre una **cuenta corriente**, su límite descubierto es 0 (no olvide definir el constructor por default).

Tarea B: Reflexione, charle con el ayudante y responda a las siguientes preguntas.

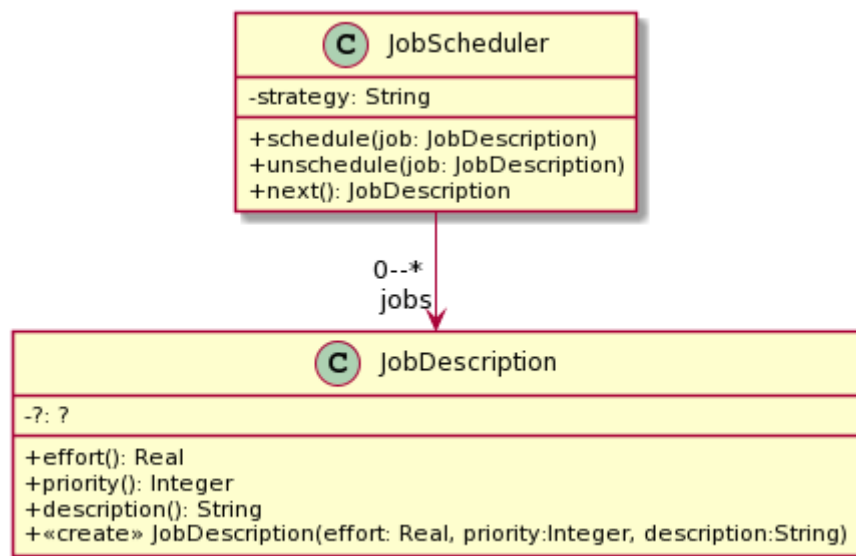
- a) ¿Por qué cree que este ejercicio se llama "Cuenta con ganchos"?

- En las implementaciones de los métodos `extraer()` y `transferirACuenta()` que se ven en el diagrama, ¿quién es `this`? ¿Puede decir de qué clase es `this`?
- ¿Por qué decidimos que los métodos `puedeExtraer()` y `extraerSinControlar` tengan visibilidad "protegido"?
- ¿Se puede transferir de una caja de ahorro a una cuenta corriente y viceversa? ¿por qué? ¡Pruébalo!
- ¿Cómo se declara en Java un método abstracto? ¿Es obligatorio implementarlo? ¿Qué dice el compilador de Java si una subclase no implementa un método abstracto que hereda?

Tarea C: Escriba los tests de unidad que crea necesarios para validar que su implementación funciona adecuadamente.

Ejercicio 10: Job Scheduler

El `JobScheduler` es un objeto cuya responsabilidad es determinar qué trabajo debe resolverse a continuación. El siguiente diseño ayuda a entender cómo funciona la implementación actual del `JobScheduler`.



- El mensaje `schedule(job: JobDescription)` recibe un job (trabajo) y lo agrega al final de la colección de trabajos pendientes.
- El mensaje `next()` determina cuál es el siguiente trabajo de la colección que debe ser atendido, lo retorna, y lo quita de la colección.

En la implementación actual del método `next()`, el `JobScheduler` utiliza el valor de la variable `strategy` para determinar cómo elegir el siguiente trabajo.

Dicha implementación presenta dos serios problemas de diseño:

- Secuencia de ifs (o sentencia `switch/case`) para implementar alternativas de un mismo comportamiento.
- Código duplicado.

Tareas:

a) **Analice el código existente**

Utilice el [código y los tests](#) provistos por la cátedra y aplique lo aprendido (en particular en relación a herencia y polimorfismo) para eliminar los problemas mencionados. Siéntase libre de agregar nuevas clases como considere necesario. También puede cambiar la forma en la que los objetos se crean e inicializan. Asuma que una vez elegida una estrategia para un scheduler no puede cambiarse.

b) **Verifique su solución con las pruebas automatizadas**

Sus cambios probablemente hagan que los tests dejen de funcionar. Corríjalos y mejórellos como sea necesario.

Ejercicio 11: El Inversor

Estamos desarrollando una aplicación móvil para que un inversor pueda conocer el estado de sus inversiones. El sistema permite manejar dos tipos de inversiones: Inversión en acciones e inversión en plazo fijo. Nuestro sistema representa al inversor y a cada uno de los tipos de inversiones con una clase.

- La clase `InversionEnAcciones` tiene las siguientes variables de instancia:

```
String nombre;  
int cantidad;  
double valorUnitario;
```

- La clase `PlazoFijo` tiene las siguientes variables de instancia:

```
LocalDate fechaDeConstitucion;  
double montoDepositado;  
double porcentajeDelInteresDiario;
```

- La clase `Inversor` tiene las siguientes variables de instancia:

```
String nombre;  
List<?> inversiones;
```

La variable `inversiones` de la clase `Inversor` es una colección con instancias de cualquiera de las dos clases de inversiones que pueden estar mezcladas.

Cuando se quiere saber cuánto dinero representan las inversiones del inversor, se envía al mismo el mensaje `valorActual()`.

Tareas:

a) **Modele e implemente**

- 1) Realice el diagrama UML de clases de la solución al problema.
- 2) Implemente en Java lo que considere necesario para que las instancias de `Inversor` entiendan el mensaje `valorActual()` teniendo en cuenta los siguientes criterios:

- El valor actual de las inversiones de un inversor es la suma de los valores actuales de cada una de las inversiones en su cartera (su colección de inversiones).
- El valor actual de un **PlazoFijo** equivale al *montoDepositado* incrementado como corresponda por el porcentaje de interés diario, desde la fecha de constitución a la fecha actual (la del momento en el que se hace el cálculo).
- El valor actual de una **InversionEnAcciones** se calcula multiplicando el número de acciones por el valor unitario de las mismas.
- Recordatorio: No olvide la inicialización.

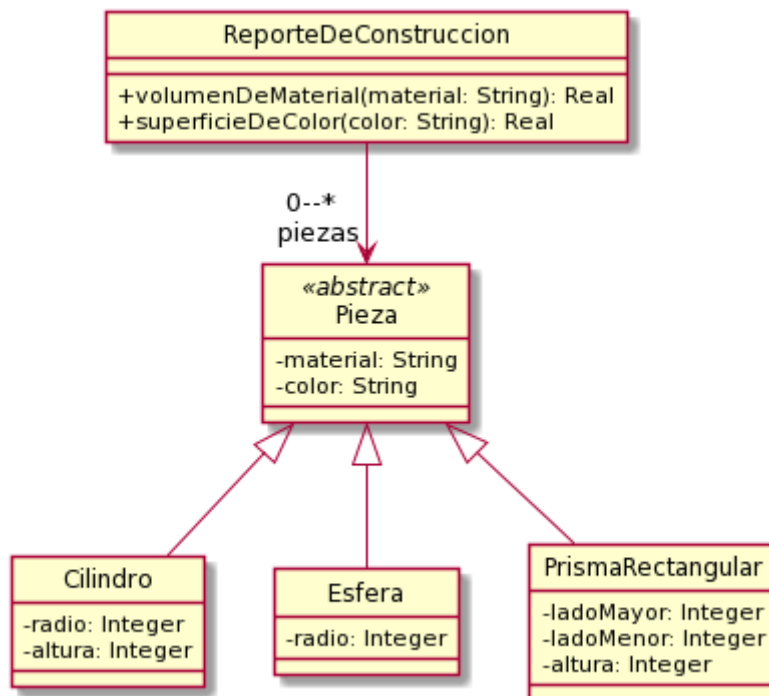
b) Pruebas automatizadas

Implemente los tests (JUnit) que considere necesarios.

Ejercicio 12: Volumen y superficie de sólidos

Una empresa siderúrgica quiere introducir en su sistema de gestión nuevos cálculos de volumen y superficie exterior para las piezas que produce. El volumen le sirve para determinar cuánto material ha utilizado. La superficie exterior le sirve para determinar la cantidad de pintura que utilizó para pintar las piezas.

El siguiente diagrama UML muestra el diseño actual del sistema. En el mismo puede observarse que un ReporteDeConstruccion tiene la lista de las piezas que fueron construidas. Pieza es una clase abstracta.



Tareas:

a) Complete el diseño e implemente

Su tarea es completar el diseño e implementarlo siguiendo las especificaciones que se exponen a continuación:

getVolumenDeMaterial(nombreDeMaterial: String)

"Recibe como parámetro un nombre de material (un string, por ejemplo 'Hierro').
Retorna la suma de los volúmenes de todas las piezas hechas en ese material"

getSuperficieDeColor(unNombreDeColor: String)

"Recibe como parámetro un color (un string, por ejemplo 'Rojo'). Retorna la suma de las superficies externas de todas las piezas pintadas con ese color".

Fórmulas

Volumen de un cilindro: $\pi * \text{radio}^2 * h$.

Superficie de un cilindro: $2 * \pi * \text{radio} * h + 2 * \pi * \text{radio}^2$

Volumen de una esfera: $\frac{4}{3} * \pi * \text{radio}^3$.

Superficie de una esfera: $4 * \pi * \text{radio}^2$

Volumen del prisma: $\text{ladoMayor} * \text{ladoMenor} * \text{altura}$

Superficie del prisma: $2 * (\text{ladoMayor} * \text{ladoMenor} + \text{ladoMayor} * \text{altura} + \text{ladoMenor} * \text{altura})$

- Para obtener π , utilizamos Math.PI
- Para elevar un número a cualquier potencia, utilizamos Math.pow(numero: double, potencia: double). Ej: $8^2 = \text{Math.pow}(8, 2)$

b) Pruebas automatizadas

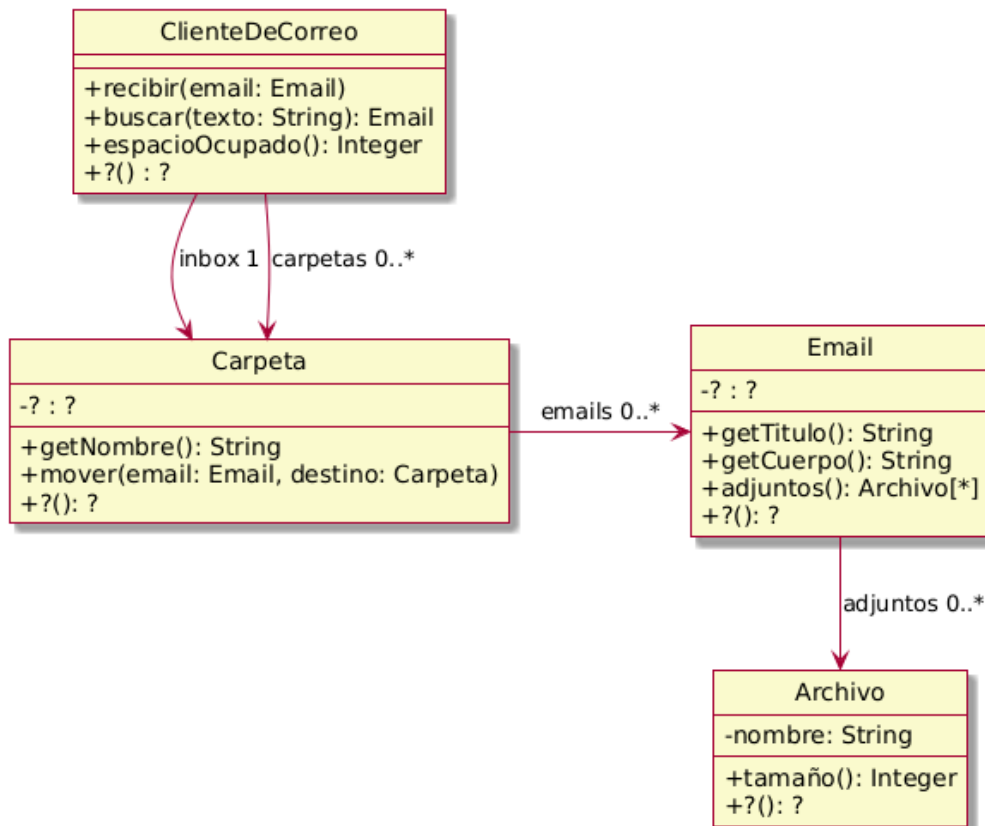
Implemente los tests (JUnit) que considere necesarios.

c) Discuta con el ayudante

Es probable que note una similitud entre este ejercicio y el de "Figuras y cuerpos" que realizó anteriormente, ya que en ambos se pueden construir cilindros y prismas rectangulares. Sin embargo las implementaciones varían. Enumere las diferencias y similitudes entre ambos ejercicios y luego consulte con el ayudante.

Ejercicio 13: Cliente de Correo

El diagrama de clases de UML que se muestra a continuación documenta parte del diseño simplificado de un cliente de correo electrónico.



Su funcionamiento es el siguiente:

- En respuesta al mensaje `#recibir`, almacena en el inbox (una de las carpetas) el email que recibe como parámetro.
- En respuesta al mensaje `#mover`, mueve el email desde una carpeta de origen a una carpeta destino (asuma que el email está en la carpeta origen cuando se recibe este mensaje).
- En respuesta al mensaje `#buscar` retorna el primer email en el Cliente de Correo cuyo título o cuerpo contienen el texto indicado como parámetro. Busca en todas las carpetas.
- En respuesta al mensaje `#espacioOcupado`, retorna la suma del espacio ocupado por todos los emails de todas las carpetas.
- El tamaño de un email es la suma del largo del título, el largo del cuerpo, y del tamaño de sus adjuntos.
- Para simplificar, asuma que el tamaño de un archivo es el largo de su nombre.

Tareas:

a) Modele e implemente

- i) Complete el diseño y el diagrama de clases UML.
- ii) Implemente en Java de la funcionalidad requerida.

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior

Ejercicio 14. Intervalo de tiempo

En Java, las fechas se representan normalmente con instancias de la clase [java.time.LocalDate](#). Se pueden crear con varios métodos "static" como por ejemplo `LocalDate.now()`.

- Investigue cómo hacer para crear una fecha determinada, por ejemplo 15/09/1972.
- Investigue cómo hacer para determinar si la fecha de hoy se encuentra entre las fechas 15/12/1972 y 15/12/2032. Sugerencia: vea los métodos permiten comparar `LocalDates` y que retornan `booleans`.
- Investigue cómo hacer para calcular el número de días entre dos fechas. Lo mismo para el número de meses y de años Sugerencia: vea el método `until`.
Tenga en cuenta que los métodos de `LocalDate` colaboran con otros objetos que están definidos a partir de enums, clases e interfaces de `java.time`; por ejemplo `java.time.temporal.ChronoUnit.DAYS`

Tareas:

a) Implemente

Implemente la clase **DateLapse** (Lapso de tiempo). Un objeto `DateLapse` representa el lapso de tiempo entre dos fechas determinadas. La primera fecha se conoce como "from" y la segunda como "to". Una instancia de esta clase entiende los mensajes:

```
public LocalDate getFrom()
    "Retorna la fecha de inicio del rango"

public LocalDate getTo()
    "Retorna la fecha de fin del rango"

public int sizeInDays()
    "retorna la cantidad de días entre la fecha 'from' y la fecha 'to'"

public boolean includesDate(LocalDate other)
    "recibe un objeto LocalDate y retorna true si la fecha está entre el from y el to del receptor y false en caso contrario".
```

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior

Ejercicio 14 b. Intervalo de tiempo

Asumiendo que implementó la clase **DateLapse** con dos variables de instancia “from” y “to”, realice otra implementación de la clase para que su representación sea a través de los atributos “from” y “sizeInDays” y coloquela en otro paquete. Es decir, debe basar su nueva implementación en estas variables de instancia solamente.

Sugerencia: Considere definir una interfaz Java para que ambas soluciones la implementen.

Los cambios en la estructura interna de un objeto sólo deben afectar a la implementación de sus métodos. Estos cambios deben ser transparentes para quien le envía mensajes, no debe notar ningún cambio y seguir usándolo de la misma forma. Tenga en cuenta que los tests que implementó en el ejercicio anterior deberían pasar **sin que se requiera realizar modificaciones**.

Ejercicio 15: Distribuidora Eléctrica

Una distribuidora eléctrica desea gestionar los consumos de sus usuarios para la emisión de facturas de cobro.

De cada usuario se conoce su nombre y domicilio. Se considera que cada usuario sólo puede tener un único domicilio en donde se registran los consumos.

Los consumos de los usuarios se dividen en dos componentes:

- **Consumo de energía activa:** tiene un costo asociado para el usuario. Se mide en kWh (kilowatt/hora).
- **Consumo de energía reactiva:** no genera ningún costo para el usuario, es decir, se utiliza solamente para determinar si hay alguna bonificación. Se mide en kVarh (kilo voltio-amperio reactivo hora).

Se cuenta con un **cuadro tarifario** que establece el precio del kWh para calcular el costo del consumo de energía activa. Este cuadro tarifario puede ser ajustado periódicamente según sea necesario (por ejemplo, para reflejar cambios en los costos).

Para emitir la factura de un cliente se tiene en cuenta **solo su último consumo registrado**.

Los datos que debe contener la factura son los siguientes:

- El usuario a quien se está cobrando.
- La fecha de emisión.
- La bonificación, sí aplica.
- El monto final de la factura: se calcula restando la bonificación al costo del consumo:
 - El costo del consumo se calcula multiplicando el consumo de energía activa por el **precio del kWh** proporcionado por el cuadro tarifario.
 - Se calcula su **factor de potencia** para determinar si hay alguna bonificación aplicable. Si el factor de potencia estimado (fpe) del último consumo del usuario es mayor a 0.8, el usuario recibe una bonificación del 10%.

El factor de potencia estimado se calcula de acuerdo a la siguiente fórmula. Para realizar las operaciones matemáticas, puede ayudarse con la clase [Math](#)

$$fpe = \frac{EnergiaActiva}{\sqrt{EnergiaActiva^2 + EnergiaReactiva^2}}$$

Tareas:

a. Analice el problema

- i) Realice el modelo de dominio detallando, cuando sea posible, para cada clase conceptual identificada:
 - 1) La categoría correspondiente a la clase
 - 2) Los atributos candidatos de la clase
 - 3) Las asociaciones entre las clases conceptuales

b. Modele e implemente

- i) Realice el diagrama de clases UML para su solución
- ii) Implemente en Java de la funcionalidad requerida.

c. Pruebas automatizadas

- i) Implemente tests automatizados utilizando JUnit para verificar su solución.

Ejercicio 16: Filtered Set

En la teoría de colecciones se explicaron algunos tipos de colecciones; en particular, el **Set** (*java.util.Set*) es una colección que no admite duplicados y no tiene índice para sus elementos.

Implemente una clase **EvenNumberSet** (conjunto de números pares). Esta clase se comporta casi exactamente igual a Set, con la diferencia que únicamente permite agregar números enteros que sean pares. Por simplicidad, considere únicamente el tipo de datos **Integer** para su solución (ignore el resto de tipos de datos numéricos).

Tenga en cuenta que la clase **EvenNumberSet** debe implementar la interface **Set<E>** de Java. Esto significa que a las variables de tipo *Set<Integer>* se les puede asignar un objeto concreto de tipo **EvenNumberSet** y luego utilizarlo enviando los mensajes que están definidos en el protocolo de **Set<E>**.

El siguiente fragmento de código ejemplifica cómo se podría usar la clase **EvenNumberSet**:

```
Set<Integer> numbers = new EvenNumberSet();
// inicialmente el Set está vacío => []
numbers.add(1); // No es par, entonces no se agrega => []
numbers.add(2); // Es par, se agrega al set => [2]
numbers.add(4); // Es par, se agrega al set => [2, 4]
numbers.add(2); // Es par, pero ya está en el set, no se agrega => [2, 4]
```

Evalúe las distintas opciones para implementar la clase **EvenNumberSet**. Para evitar reinventar la rueda, considere reutilizar alguna de las clases existentes en Java que ofrezcan funcionalidades similares.

Tareas:

- a. Investigue qué clases se pueden utilizar para implementar la clase **EvenNumberSet**. Consulte la [documentación de Set](#).
- b. Explique brevemente cómo propone utilizar las clases investigadas anteriormente para implementar su solución. Por ejemplo:
 - “Se debe subclasificar una determinada clase y redefinir un método para que haga lo siguiente”
 - “Se debe crear una nueva clase que contenga un objeto de un determinado tipo al cual se le delegará esta responsabilidad”
- c. Implemente en Java las alternativas que haya propuesto.
- d. Implemente tests automatizados utilizando JUnit para verificar sus implementaciones.
- e. Compare las soluciones y liste las ventajas y desventajas de cada una.

Ejercicio 17. Alquiler de propiedades

Se desea diseñar e implementar una plataforma para gestión de reservas de propiedades que llamaremos OOBnB. En la misma, los usuarios pueden gestionar sus inmuebles para su alquiler así como también realizar reservas sobre estos.

De los usuarios se conoce el nombre, la dirección y el DNI. Cada usuario posee propiedades que desea alquilar, de las cuales se guarda la dirección, un nombre descriptivo y el precio que se desea cobrar por noche. Además, los usuarios pueden realizar reservas sobre cualquiera de las propiedades disponibles.

Nos piden implementar la siguiente funcionalidad:

- **Consultar la disponibilidad de una propiedad en un período específico:** dada una propiedad, una fecha inicial y una fecha final, se debe determinar si la propiedad está disponible el período indicado.
- **Crear una reserva:** Un usuario puede realizar una reserva para un período de tiempo determinado. Si la propiedad está disponible, se crea la reserva y la propiedad pasa a estar ocupada durante ese período. Si no lo está, la reserva no será creada.
- **Calcular el precio de una reserva:** Dada una reserva, se debe poder calcular su precio. El mismo se obtiene multiplicando la cantidad de noches por el precio por noche.
- **Cancelar una reserva:** Se debe permitir cancelar una reserva. En este caso, la propiedad pasa a estar disponible durante el período de tiempo indicado en la reserva. Esta operación sólo es permitida si el período de la reserva no está en curso.
- **Calcular los ingresos de un propietario:** Se debe calcular la retribución a un propietario, la cual es el 75% de la suma de precios totales de las reservas incluidas en un período específico de tiempo.

Notas sobre el diseño e implementación:

Para el manejo de los períodos de reserva se sugiere añadir un nuevo método a la interfaz **DateLapse** definida en el ejercicio anterior (**ejercicio de Intervalos de tiempo**).

A modo de sugerencia, la especificación del mismo puede ser la siguiente:

```
/**
    Retorna true si el periodo de tiempo del receptor se superpone con el
    recibido por parámetro
**/
public boolean overlaps (anotherDateLapse: DateLapse)
```

Tareas:

1. Realice el diagrama del modelo conceptual.
2. Realice el diagrama de clases UML
3. Implemente en Java de la funcionalidad requerida.
4. Implemente en JUnit pruebas automatizadas en donde:
 - a. Se muestre cómo verificar la disponibilidad de una fecha para una propiedad que no tenga reservas en ese período.
 - b. Se muestre cómo verificar la disponibilidad de una fecha para una propiedad en la que haya reservas en ese período.
 - c. Se muestre cómo se reserva una propiedad para un período en particular.
 - d. Se muestre cómo cancelar una reserva creada.

Ejercicio 18. Políticas de cancelación

A la implementación del ejercicio anterior se quiere extender la funcionalidad de cancelar una reserva de manera tal que calcule el monto que será reembolsado (devuelto) al inquilino. Para hacer esto posible, las propiedades deben conocer una política de cancelación que se define al momento de crearla. Esta política puede ser una de las siguientes: flexible, moderada, o estricta y puede cambiarse en cualquier momento.

Al momento de reembolsar, las políticas se comportan de la siguiente manera:

- a) **Política de cancelación flexible:** reembolsa el monto total sin importar la fecha de cancelación (que de todas maneras debe ser anterior a la fecha de inicio de la reserva).
- b) **Política de cancelación moderada:** reembolsa el monto total si la cancelación se hace hasta una semana antes y 50% si se hace hasta 2 días antes.
- c) **Política de cancelación estricta:** no reembolsará nada (0, cero) sin importar la fecha tentativa de cancelación.

Actualice su diseño, implementación y tests de acuerdo a los nuevos requerimientos.

Ejercicio 19. Servicio de envíos de paquetes

Una **empresa de envíos de paquetes** ofrece a sus clientes distintos servicios, como envíos locales, interurbanos e internacionales. Los envíos locales son envíos dentro de la misma

ciudad y cuentan con una opción de entrega rápida. Los envíos interurbanos son envíos entre ciudades. Los envíos internacionales son envíos a destinos fuera del país.

De cada envío se registra la fecha en la cual se realiza el despacho, la dirección de origen, la dirección de destino y el peso expresado en gramos. Para los interurbanos además, la distancia expresada en km.

La empresa trabaja con dos tipos de clientes: **personas físicas**, que son individuos, y **clientes corporativos**, empresas que tienen un volumen alto de envíos. De las personas físicas, se conoce el nombre, dirección y DNI. De los clientes corporativos se conoce nombre de la empresa, dirección y CUIT.

Nos piden implementar la siguiente funcionalidad:

Agregar un envío para un cliente: dado un cliente y un envío, se agrega ese envío al cliente indicado.

Monto a pagar por los envíos realizados dentro de un período. Se indica el cliente para el cual se quiere calcular el monto y las fechas de inicio y fin del período a considerar. Para calcular el monto total a pagar, se suma el costo de todos los envíos despachados durante el período especificado.

- Los envíos locales tienen un costo fijo de \$1000 para las entregas estándar y \$500 adicional por entrega rápida .
- Los envíos interurbanos tienen un costo que depende de la distancia entre el origen y el destino (utilice \$20 para menos de 100 km por cada gramo de peso, \$25 para distancias entre 100 km y 500 km por gramo de peso, y \$30 para distancias de más de 500 km por gramo de peso).
- Los envíos internacionales tienen un costo que depende del país destino y del peso del paquete. Por ahora, utilice \$5000 para cualquier destino y \$10 por gramo de peso para envíos de hasta 1 kg y \$12 para envíos de más de 1 kg.

Los envíos efectuados por personas físicas tienen un 10% de descuento.

Tareas:

a) Modele e implemente

- i) Diagrama de clases UML.
- ii) Implemente en Java la funcionalidad requerida.

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior.

c) Es probable que los montos utilizados para los cálculos le hayan quedado fijos dentro del código (hardcoded). Piense qué pasaría si al calcular el monto a pagar se proveyera (como un parámetro más) el "cuadro tarifario". ¿Cómo sería ese objeto? ¿Qué responsabilidad le podría delegar? ¿Cómo haríamos para tener montos diferentes para los distintos países en los envíos internacionales según los pesos de los envíos?

Ejercicio 20. Liquidación de haberes

Estamos desarrollando una aplicación para realizar la liquidación de haberes de los empleados de una empresa. De estos se conoce nombre, apellido, CUIL, fecha de nacimiento, si tiene hijos a cargo y los contratos de trabajo que tiene con la empresa.

Los contratos de los empleados tienen la fecha de inicio del contrato, la fecha de fin (si corresponde) y algunos valores adicionales dependiendo del tipo de contrato. Hay dos tipos de contratos:

1. Si el contrato es "por horas", se indica el valor-hora acordado, y el número de horas que trabajará por mes. También se indica la fecha de fin del contrato.
2. Si el contrato es "de planta", se indica el sueldo mensual acordado, el monto acordado por tener cónyuge a cargo, y el monto acordado por tener hijos a cargo. Estos contratos no tienen fecha de fin (nunca se vencen).

Pueden existir varios contratos para un mismo empleado; sin embargo, un empleado solo puede tener un único contrato activo (no vencido) a la vez. El contrato activo para el caso de contrato "de planta" es el único contrato vigente. Para un contrato "por horas", se considera activo aquel cuya fecha de fin sea posterior a la fecha actual.

Nos piden implementar:

Generar recibo de sueldo para un empleado: Para generar el recibo de sueldo de un empleado se tiene en cuenta **solo su contrato vigente**. El recibo de sueldo debe contener la siguiente información: el nombre, apellido, CUIL y antigüedad en la empresa del empleado al que pertenece el recibo; la fecha en la que fue generado el recibo; y el monto total que le corresponde cobrar al empleado según el contrato vigente.

El monto se calcula en dos pasos, el sueldo básico y un adicional por antigüedad. El básico se calcula de la siguiente forma:

1. Si su contrato es por horas fijas, el monto a cobrar es el valor-hora acordado multiplicado por el número de horas que trabaja por mes.
2. Si su contrato es de planta, el monto a cobrar es el sueldo mensual acordado, más el monto acordado por tener cónyuge a cargo (si es que tiene cónyuge a cargo), más el monto acordado por tener hijos a cargo (si es que tiene hijos a cargo).

El adicional por antigüedad se calcula como un porcentaje del básico. La política de la empresa determina que el porcentaje se aumente automáticamente cuando se alcanza cierta antigüedad, en función de esta escala: 5 años 30%, 10 años 50%, 15 años 70%, 20 años 100%. Tenga en cuenta que la antigüedad de un empleado se calcula como **la suma de las duraciones de cada uno de los contratos registrados**.

Tareas:

a) Modele e implemente

- i) Diagrama de clases UML.
- ii) Implementación en Java la funcionalidad requerida.

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior.

Ejercicio 21. Bag

Primera parte

Un [Map](#) en Java es una colección que asocia objetos que actúan como claves (*keys*), a otros objetos que actúan como valores (*values*). En otros lenguajes también se llaman Dictionary (Diccionario). Cada clave está vinculada a un único valor; no pueden existir claves duplicadas en un mapa, aunque sí podrían haber valores repetidos. A los pares clave-valor se los denomina entradas (*entries*).

Para definir un Map, es necesario indicar el tipo que tendrán sus claves y valores: por ejemplo, una variable de tipo `Map<String, Integer>` define un mapa en donde sus claves son de tipo *String*, y sus valores de tipo *Integer*. Observe que `Map<K, V>` es una interfaz.

Tareas:

- a. Lea la [documentación de Map en Java](#) y responda:
 - a. ¿Qué implementaciones provee Java para utilizar un Map? ¿Cuáles de ellas son destinadas a uso general?
 - b. Investigue cómo consultar si un mapa contiene una determinada clave (key). Explique qué métodos deben implementar las claves para que esto funcione correctamente
 - c. ¿Con qué método se puede recuperar el objeto asociado a una clave? ¿Qué pasa si la clave no existe en el mapa?
 - d. Investigue cómo agregar claves y valores a un mapa. ¿Qué pasa si la clave ya se encontraba en el mapa? ¿Permite agregar claves y/o valores nulos?
 - e. Determine cómo se pueden eliminar claves y valores de un mapa. ¿Es necesario controlar la presencia de alguno de ellos?
 - f. Investigue cómo reemplazar un valor en un mapa
 - g. Teniendo en cuenta los métodos `keySet()`, `values()` y `entrySet()`, explique de qué formas se puede iterar un mapa ¿Es posible utilizar streams?
- b. Para practicar los mensajes investigados anteriormente, escriba un test de unidad que contenga lo siguiente:
 - a. Cree un map un `Map<String, Integer>`, y agregue las tuplas ("Lionel Messi", 111), ("Gabriel Batistuta", 56), ("Kun Agüero", 42)
 - b. Elimine la entrada con clave "Kun Agüero"
 - c. Messi hizo 112 goles a día de la fecha; actualice la cantidad de goles
 - d. Intente repetir la clave "Gabriel Batistuta" y verifique que no es posible.
 - e. Obtenga la cantidad total de goles
- c. Como se mencionó, cualquier objeto puede actuar como clave. Es decir, pueden ser instancias de clases definidas por el programador. Modele e implemente la clase Jugador con apellido y nombre. Escriba otro test de unidad similar al de la tarea 2, pero utilizando `Map<Jugador, Integer>`

Segunda parte

Un **Bag** (bolsa) es una colección que permite almacenar elementos sin ningún orden específico y admite elementos repetidos. Este objeto requiere un buen tiempo de respuesta para conocer la cardinalidad de sus elementos, y por esa razón almacena la **cardinalidad** de cada elemento (cantidad de veces que fue agregado en la bolsa). Por ejemplo, si agregamos 3 veces un objeto en la bolsa, y luego eliminamos 1 referencia, la cardinalidad de ese objeto en la bolsa es 2.

El protocolo de la interface Bag<T> es:

```
public interface Bag<T> extends Collection<T> {
    /**
     * Agrega un elemento al Bag, incrementando en 1 su cardinalidad.
     */
    @Override
    boolean add(T element);

    /**
     * Devuelve la cardinalidad del elemento. Si el elemento no está en el Bag,
     * devuelve 0.
     */
    int occurrencesOf(T element);

    /**
     * Elimina una referencia del elemento del Bag. Si el elemento no está en
     * el Bag, no hace nada.
     */
    void removeOccurrence(T element);

    /**
     * Elimina el elemento del Bag. Si el elemento no está en el Bag, no hace
     * nada
     */
    void removeAll(T element);

    /**
     * Devuelve el número total de elementos en el Bag, es decir, la suma de
     * todas las cardinalidades de todos sus elementos.
     */
    @Override
    int size();
}
```

Observe que la interfaz Bag<T> extiende Collection<T>.

Tareas:

1. Liste los métodos que debe contener una clase que implementa la interface Bag<T>.
2. Explique cómo implementaría un **Bag<T>** usando composición con un **Map<K, V>**.
¿De qué tipo tendrían que ser las claves y valores del Map?
3. Implemente la interfaz Bag<T>, utilizando **AbstractCollection<T>** como superclase, y compóngala con un **Map<T, V>**. Para simplificar la implementación utilice la clase BagImpl que se encuentra en este [link](#).
4. Discuta con un ayudante:
 - a. ¿Cuáles son los beneficios de utilizar **AbstractCollection** como superclase para implementar el Bag?
 - b. ¿Qué ventajas tiene componer con un Map para implementar el Bag?
 - c. En lugar de componer con un Map, ¿es posible extenderlo para poder implementar el Bag? ¿Qué diferencias tendría esa solución con respecto a la planteada en este ejercicio?
 - d. ¿Para qué cree que podría ser útil un objeto Bag?

Ejercicio 22. Estadísticas del Cliente de Correo

Extienda el [Ejercicio 13: Cliente de Correo](#):

Nos piden implementar la siguiente funcionalidad:

- cantidad de emails que tiene una carpeta
- cantidad total de emails en el cliente de correo: considerando todas las carpetas existentes.
- cantidad de mails por categoría: para cada carpeta se debe calcular y retornar en un solo objeto, la cantidad de emails categorizados por tamaño siguiendo el siguiente criterio
 - Pequeño: el email tiene un tamaño entre 0 y 300
 - Mediano: el email tiene un tamaño entre 301 y 500
 - Grande: el email tiene un tamaño mayor a 501

Tareas:

a) Modele e implemente

- i) Modifique el diagrama de clases UML según lo necesario
- ii) Implemente en Java los cambios solicitados

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior.

Ejercicio 23. Mercado de Objetos

Queremos programar en objetos una versión simplificada de un mercado on-line similar a

e-Bay o MercadoLibre. En esta plataforma, los vendedores registrados pueden publicar productos disponibles a la venta, y los clientes pueden realizar pedidos para comprar esos productos.

De los vendedores se conoce su nombre, dirección y los productos que ofrece para la venta, de los cuales se guarda nombre, categoría, precio y cantidad de unidades disponibles.

De los clientes se conoce su nombre, dirección y los pedidos que haya solicitado.

La plataforma ofrece diferentes opciones de pago, actualmente: "al contado" o "6 cuotas". A futuro podrían agregarse otras formas de pago.

También existen diferentes formas de envío: "retirar en el comercio", "retirar en sucursal del correo", ó "express a domicilio". A futuro podrían agregarse otras formas de envío.

Nos piden implementar la siguiente funcionalidad:

Crear un pedido para un cliente: dado un cliente, una forma de pago, una forma de envío, un producto y la cantidad solicitada, se verifica si hay unidades disponibles del producto: si es así, se crea el pedido y se descuentan las unidades indicadas del producto. No hace nada si no hay suficientes unidades disponibles.

Conocer la cantidad de productos pedidos por categoría: dado un cliente, se desea conocer cuántos productos por categoría ha pedido en la plataforma. Ejemplos de categorías son: "Electrodomésticos", "Computadores", "Hogar", etc.

Calcular el costo total de un pedido. Dado un pedido, se retorna su costo total que se calcula de la siguiente forma: (precio final en base a la forma de pago seleccionada) + (costo de envío en base a la forma de envío seleccionada).

- Si la forma de pago es "al contado", el precio final es el que se indica en el producto
- Si la forma de pago es "6 cuotas", el precio final se incrementa en un 20%
- Si la forma de envío es "retirar en el comercio" no hay costo adicional de envío.
- Si la forma de envío es "retirar en sucursal del correo" el costo es de \$3000.
- Si la forma de envío es "express a domicilio" el costo es \$0.5 por Km de distancia entre la dirección del vendedor y del cliente. Asuma que existe una clase CalculadoraDeDistancia, cuyas instancias entienden el mensaje #distanciaEntre que recibe dos direcciones y retorna la distancia en Km entre ellas. Por ahora trabaje con una implementación propia para pruebas que siempre retorna 100 (sin importar las direcciones recibidas).

Tareas:

a) Modele e implemente

- i) Diagrama de clases UML.
- ii) Implemente en Java la funcionalidad requerida.

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior.

Ejercicio 24. PoolCar

PoolCar es una aplicación que permite a sus usuarios compartir costos en viajes de larga distancia.

De cada usuario se conoce su nombre y saldo. El saldo les permitirá pagar los viajes que realicen. Hay dos tipos de usuarios: conductores y pasajeros. El sistema tiene una restricción: un usuario se registra a la aplicación como conductor o como pasajero, y no puede cambiar su rol. Cada conductor tiene un vehículo y solamente puede viajar en él. De cada vehículo se conoce: dueño/conductor, descripción, capacidad de pasajeros (incluido el conductor), año de fabricación y el valor de mercado.

Cuando se registra un viaje, se hace guardando: origen, destino, costo total, vehículo y fecha de viaje y se agrega al dueño del vehículo como pasajero del viaje. El conductor compartirá los gastos del viaje con los demás pasajeros.

Los usuarios pasajeros pueden registrarse a los viajes siempre y cuando haya capacidad en el vehículo, se registren al menos dos días antes de la fecha del mismo y no tengan su saldo en rojo (menos de 0).

Al procesar un viaje se debe descontar el saldo correspondiente a los integrantes (conductor y pasajeros) del mismo. El costo total del viaje se reparte en partes iguales entre todos los integrantes que participaron. Sin embargo, al momento de procesar un viaje, se realiza una bonificación que se calcula de manera distinta para los conductores que para los pasajeros. Del monto correspondiente al conductor, se le bonifica el 0.1% del valor del vehículo utilizado para el viaje. Del monto correspondiente a un pasajero, se bonifica \$500 si el usuario hizo un viaje en el pasado. Es posible que, luego de procesar el viaje, el saldo de un usuario quede en rojo (saldo < 0).

Nos piden implementar la siguiente funcionalidad:

Cargar saldo para un usuario: dado un usuario y un monto, incrementa el monto indicado en el saldo en la cuenta del usuario. Se cobra una comisión (extra sobre el saldo). Para el caso de los conductores, la comisión es siempre del 1% si el auto tiene menos de 5 años. Si no, es del 10%. En el caso de un pasajero, se cobra una comisión del 10% solo si no realizó ningún viaje los últimos 30 días.

Dar de alta un viaje desde un origen a un destino: se provee el origen, el destino, el costo total, el vehículo con el que se realizará el viaje y la fecha.

Registrar pasajero para un viaje: dado un pasajero y un viaje, registra ese pasajero para que pueda compartir ese viaje, con las reglas explicadas anteriormente.

Procesar un viaje: permite procesar un viaje siguiendo la especificación anterior.

Tareas:

a) **Modele e implemente**

- i) Diagrama de clases UML.
- ii) Implementación en Java de la funcionalidad requerida.
- b) Pruebas automatizadas**
 - i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
 - ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior.

Ejercicio 25. Veterinaria

Se debe desarrollar una plataforma para una veterinaria donde se permita registrar los servicios médicos provistos por un médico veterinario a una mascota.

De los médicos veterinarios se conoce el nombre, fecha de ingreso a la Veterinaria y honorarios a cobrar por atención. De las mascotas se conoce su nombre, fecha de nacimiento y especie (por simplicidad un String).

Los servicios médicos pueden ser consultas médicas, vacunaciones o servicio de guardería. Solo las consultas médicas y la vacunación tienen intervención de un médico veterinario.

- De las consultas médicas se conoce el médico, la mascota y la fecha de atención.
- De las vacunaciones se conoce el médico, la mascota, el nombre de la vacuna a aplicar y su costo.
- De los servicios de guardería se conoce la mascota y la cantidad de días.

Nos piden implementar la siguiente funcionalidad:

Dar de alta una consulta médica para una mascota: Dado un médico y una mascota, se crea una consulta médica para dicha mascota considerando la fecha actual como fecha de atención y la retorna.

Dar de alta una vacunación para una mascota: Dado un médico, una mascota, el nombre de la vacuna a aplicar y su costo, se crea la vacunación de dicha mascota considerando la fecha actual como fecha de vacunación y la retorna.

Dar de alta un servicio de guardería para una mascota: Dada una mascota y la cantidad de días, se crea el servicio de guardería para dicha mascota considerando la fecha actual como inicio del período y lo retorna.

Calcular el costo de un servicio: Los costos de los servicios se calculan de la siguiente manera:

- La consulta médica se cobra calculando la suma de los siguientes valores:
 - honorarios del médico veterinario que interviene
 - adicional de materiales descartables (\$300).
 - adicional por atención en día domingo (\$200).
 - adicional por antigüedad del médico (\$100 por año de antigüedad).
- La vacunación se cobra calculando la suma de los siguientes valores:
 - honorarios del médico veterinario que interviene
 - adicional de materiales descartables (\$500).

- adicional por atención en día domingo (\$200).
- el costo de la vacuna utilizada
- La guardería se cobra según un costo diario de \$500 y, si la mascota utilizó previamente 5 o más servicios, se le aplica un descuento del 10%.

Determinar la recaudación generada por una mascota en una fecha dada: dada una fecha, se debe retornar el monto recaudado en todos los servicios recibidos por esa mascota en dicha fecha.

Tareas:

a) Modele e implemente

- i) Diseño de su solución en un diagrama de clases UML.
- ii) Implementación en Java de la funcionalidad requerida.

b) Pruebas automatizadas

- i) Diseñe los casos de prueba teniendo en cuenta los conceptos de valores de borde y particiones equivalentes vistos en la teoría.
- ii) Implemente utilizando JUnit los tests automatizados diseñados en el punto anterior.